

AI Computing Infrastructure

DS5110: Big Data Systems

Summer 2026

Lecture 8

Yue Cheng



UNIVERSITY
of VIRGINIA

Some material taken/derived from:

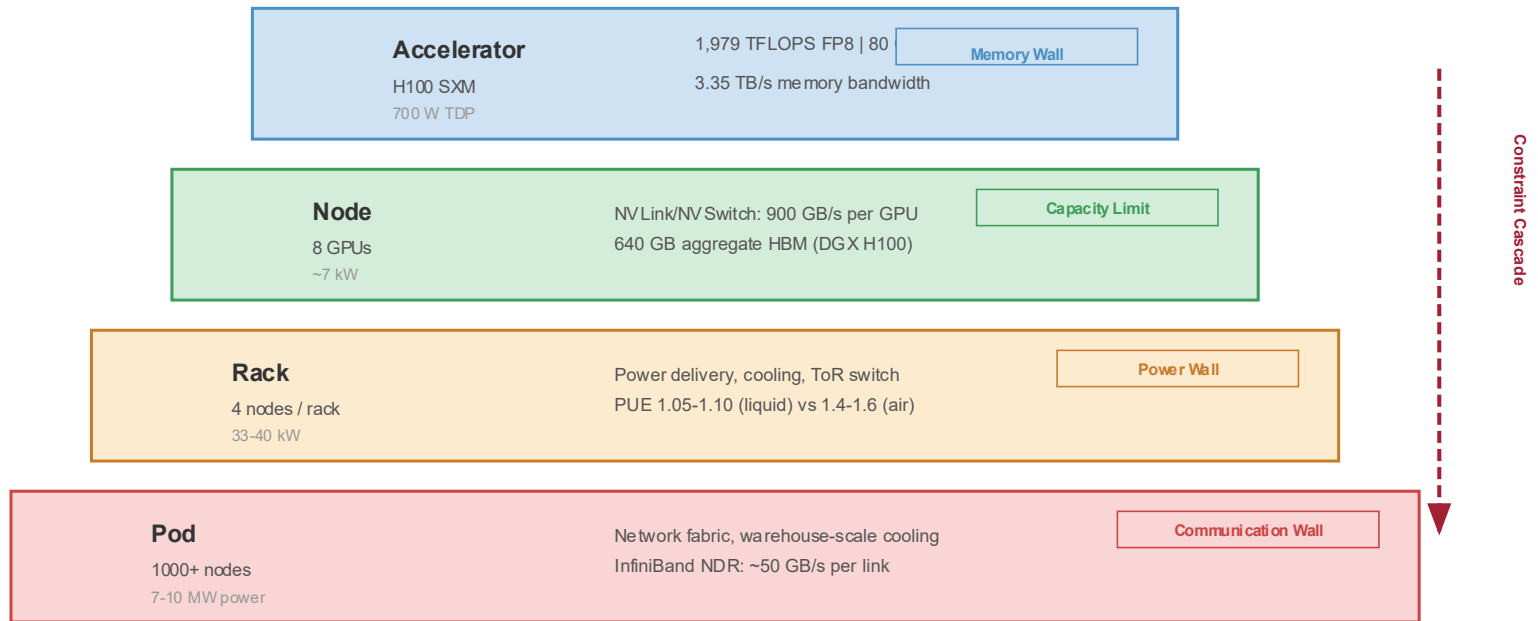
- Harvard ML Systems by Vijay Janapa Reddi

@ 2026 released for use under a [CC BY-SA](#) license.

AI infrastructure stack

The Infrastructure Stack: From Transistor to Datacenter

Running example: 175B model → 350 GB weights (FP16)



Each constraint at one level creates the engineering motivation for the next level.

Key principle: Each constraint at one level creates the engineering motivation for the next level.

Three walls across the stack

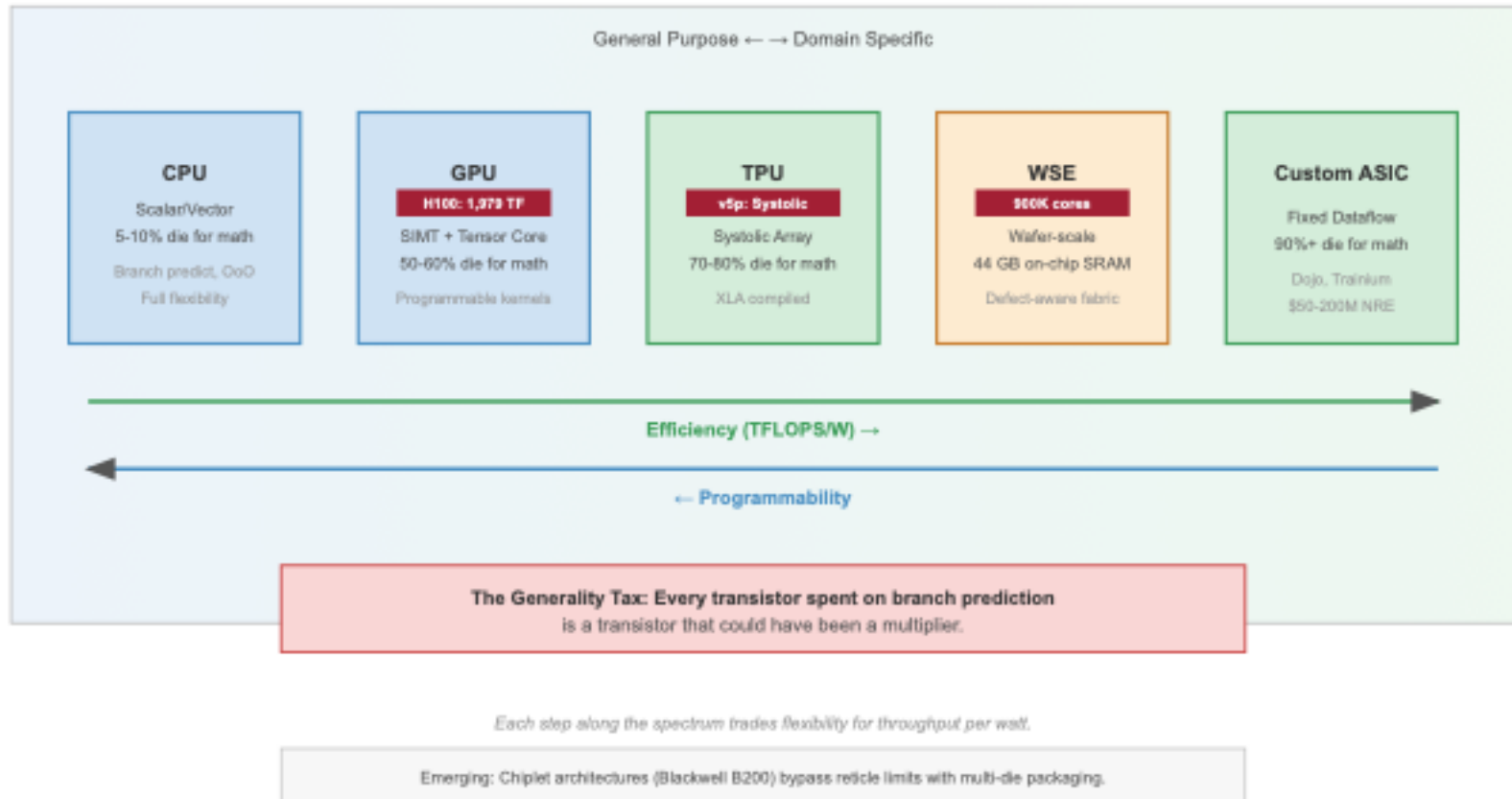
Level	Memory Wall	Power Wall	Communication Wall
Accelerator	HBM bandwidth	700 W TDP	On-die mesh
Node	640 GB aggregate	7 kW per node	NVLink: 900 GB/s
Rack	PCIe to host DRAM	33–40 kW/rack	ToR switch
Pod	Distributed storage	7–10 MW	IB: 50 GB/s

Running example: 175B model → 350 GB weights (FP16) → 2.1 TB training state → does not fit in one chip

Accelerators

The accelerator spectrum

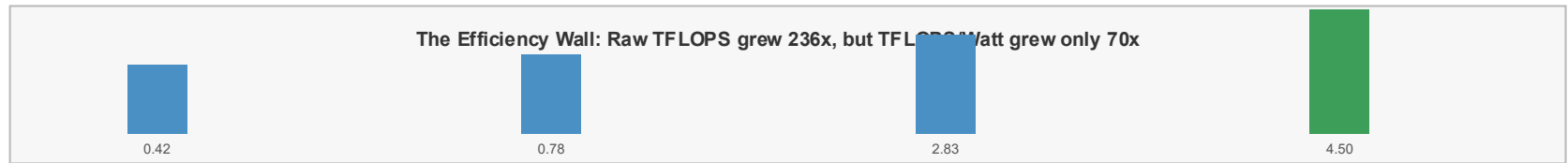
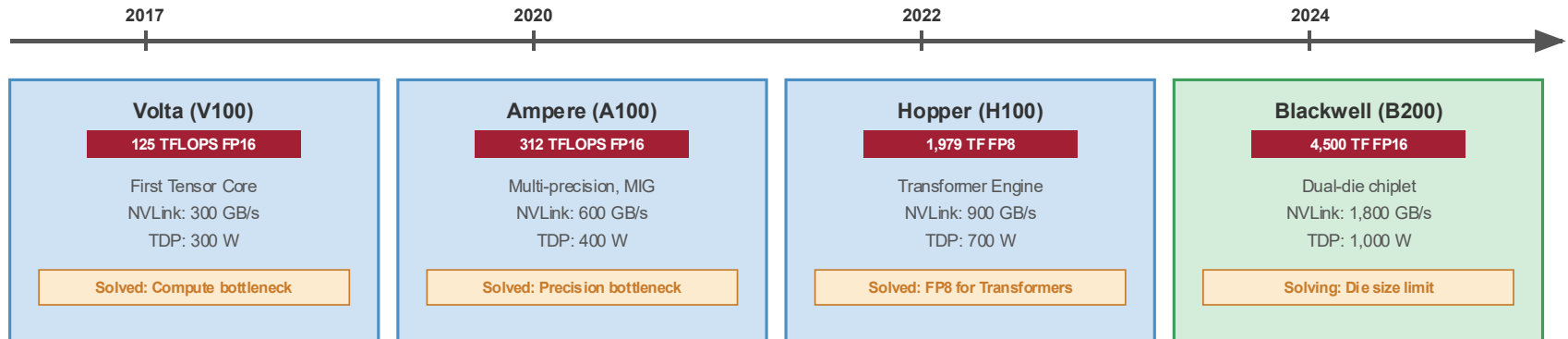
The Accelerator Spectrum: Programmability vs. Efficiency



The generality tax: Every transistor spent on a branch prediction is a transistor that could have been a multiplier.

GPU architecture evolution

GPU Architecture Evolution: Bottleneck-Driven Design



Pattern: Each generation addresses the bottleneck exposed by the previous one.

Hardware useful lifetime shrinking: V100 (3 yr) → A100 (2 yr) → H100 (~2 yr) → Blackwell

GPU architects profile production ML workloads to identify the next bottleneck. The workload drives the silicon.

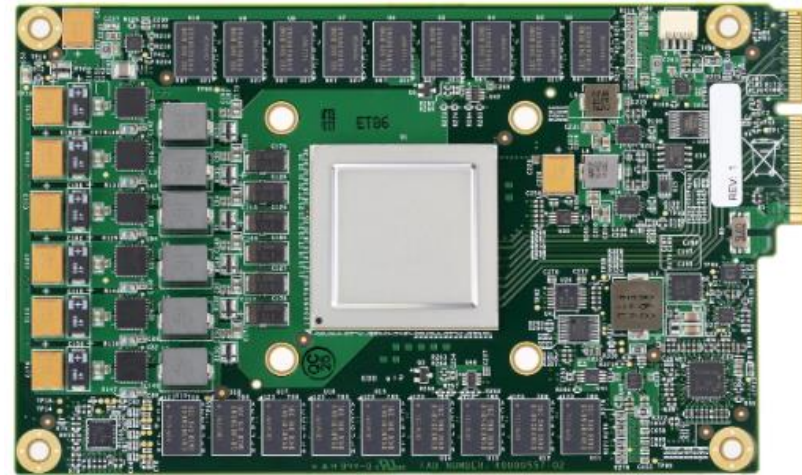
GPU specs: Four generations

Arch.	Year	Peak TF	TDP	NVLink	TF/W
Volta (V100)	2017	125	300 W	300 GB/s	0.42
Ampere (A100)	2020	312	400 W	600 GB/s	0.78
Hopper (H100)	2022	1,979	700 W	900 GB/s	2.83
Blackwell (B200)	2024	4,500	1,000 W	1,800 GB/s	4.50

The efficiency wall: Raw TFLOPS grew 236x (P100 → B200), but TFLOPS/Watt grew only 70x. The gap is what cooling and power infrastructure must absorb.

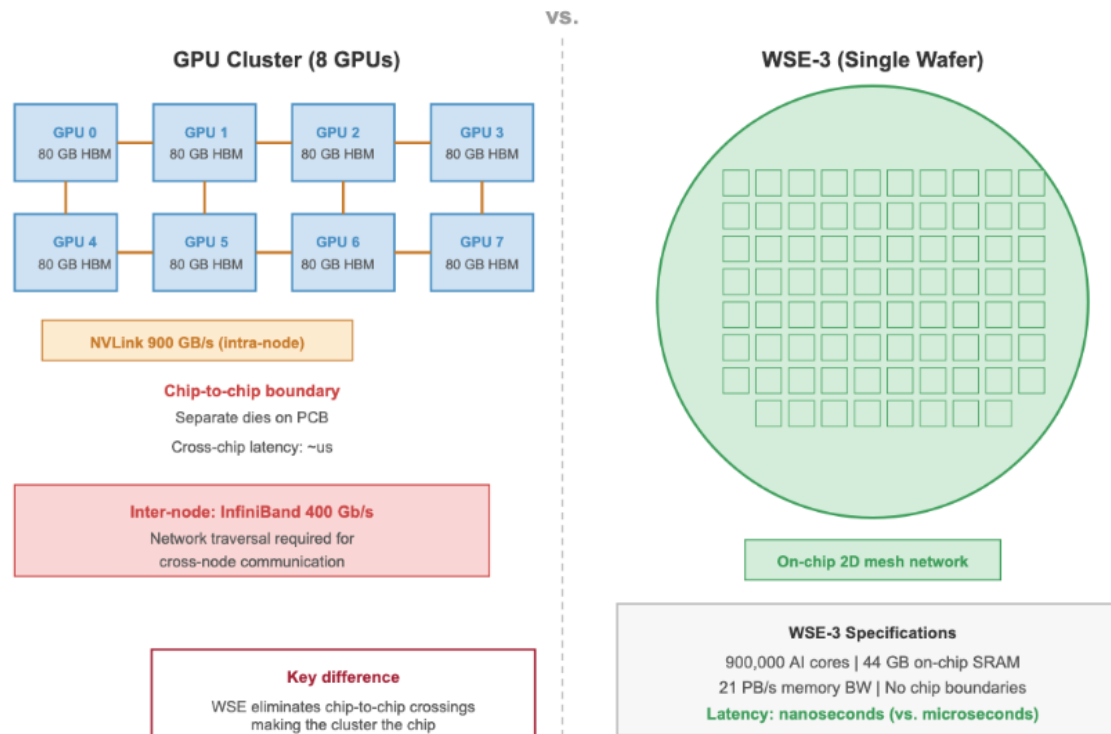
Google TPUs

- In 2013, Google engineers projected if every user spoke to their Android phone for just 3 min per day using voice search, Google would need to **double its datacenter compute capacity**
- Over **90%** of inference cycles were spent on matrix multiplications in NNs
- GPUs are powerful but **cost prohibitive**
- General-purpose SIMT processors carried the **generality tax**



TPU printed circuit board. It can be inserted into the slot for a SATA disk in a server.

Wafer-scale engine (WSE)



- A WSE is a single, continuous piece of silicon the size of a dinner plate
- WSE-3 contains 900,000 AI-optimized cores and 44 GB of on-chip SRAM, all connected by a silicon fabric that delivers 21 PB/s of memory bandwidth

Beyond GPUs: TPUs, wafer-scale

Accelerator	BF16 TF (sparse)	Mem BW	Interconnect	TDP
H100 (NVIDIA)	1,979	3,350 GB/s	NVLink: 900 GB/s	700 W
TPU v4 (Google)	275	1,200 GB/s	ICI: 4,800 Gbps	170 W
CS-2 (Cerebras)	1,840	20 PB/s (on-chip)	N/A (single wafer)	23 kW

Selection criteria: (1) Workload pattern (dense vs. sparse), (2) Ecosystem (CUDA vs. JAX vs. custom), (3) Interconnect topology, (4) Supply chain and cost.

Lock-in risk: GPU/CUDA dominates because of ecosystem, not hardware superiority. TPU ICI is 6x IB bandwidth.

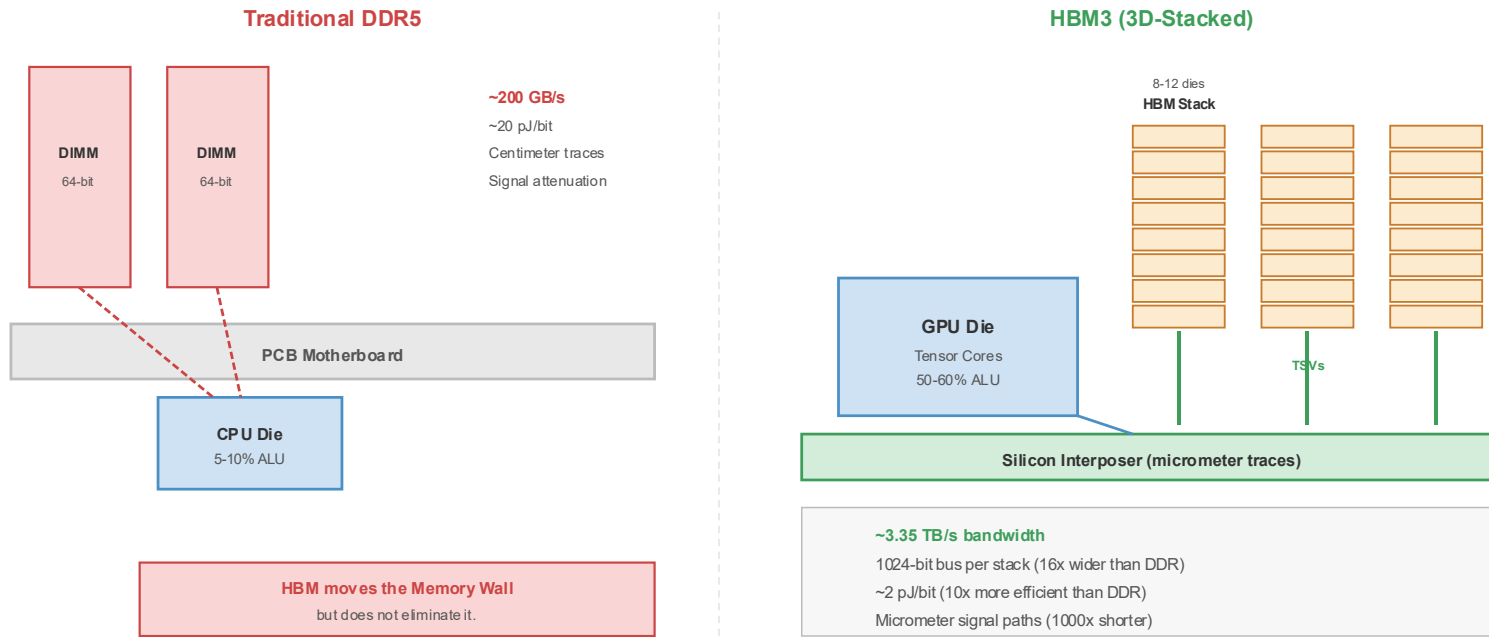
Memory wall

The memory well

The paradox: Faster logic idles more, waiting for data.

Metric	Value
DDR5 BW	~200 GB/s
HBM3 BW	3,350 GB/s
Ratio	50x
DDR5 energy	20 pJ/bit
HBM3 energy	2 pJ/bit

HBM 3D-Stacked Architecture vs. Traditional DDR



HBM: 3D stacking replaces centimeters of PCB trace with micrometers of vertical silicon.

Key: HBM moves the memory wall but does not eliminate it.

Token latency: The physics

Scenario: One token of Llama-3 70B on H100

Component	Calculation	Time	Share
Compute	$2 \times 70\text{B} / 989 \text{ TF}$	0.14 ms	<1%
Memory	$140 \text{ GB} / 3.35 \text{ TB/s}$	41.8 ms	> 99%
		$T_{\text{token}} \approx$	41.9 ms

The processor spends more than **99%** of this idealized token step waiting for **data**. Even a hypothetical chip with infinite compute would generate tokens only negligibly faster. **Memory bandwidth is the binding constraint.**

Question: Memory-bound or compute-bound?

A team trains a 175B model at batch size 2,048.
Another team serves the same model at batch size 1.

Which workload benefits more from buying a faster GPU (more TFLOPS)?

Roofline

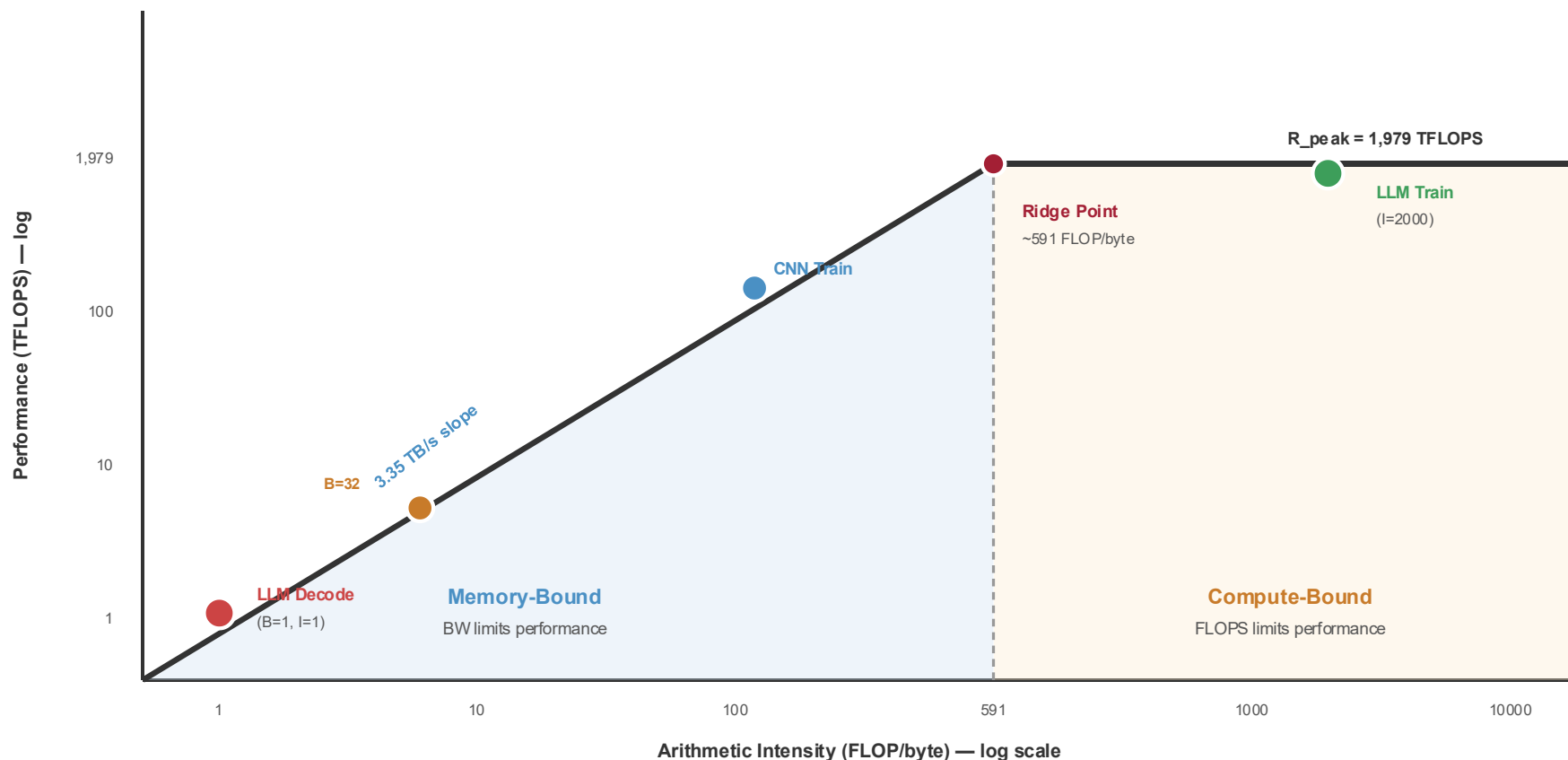
The Roofline model

$$\text{Achievable FLOPS} = \min(R_{peak}, BW \times I)$$

where I = FLOP/byte (arithmetic intensity)

Roofline: H100 landscape

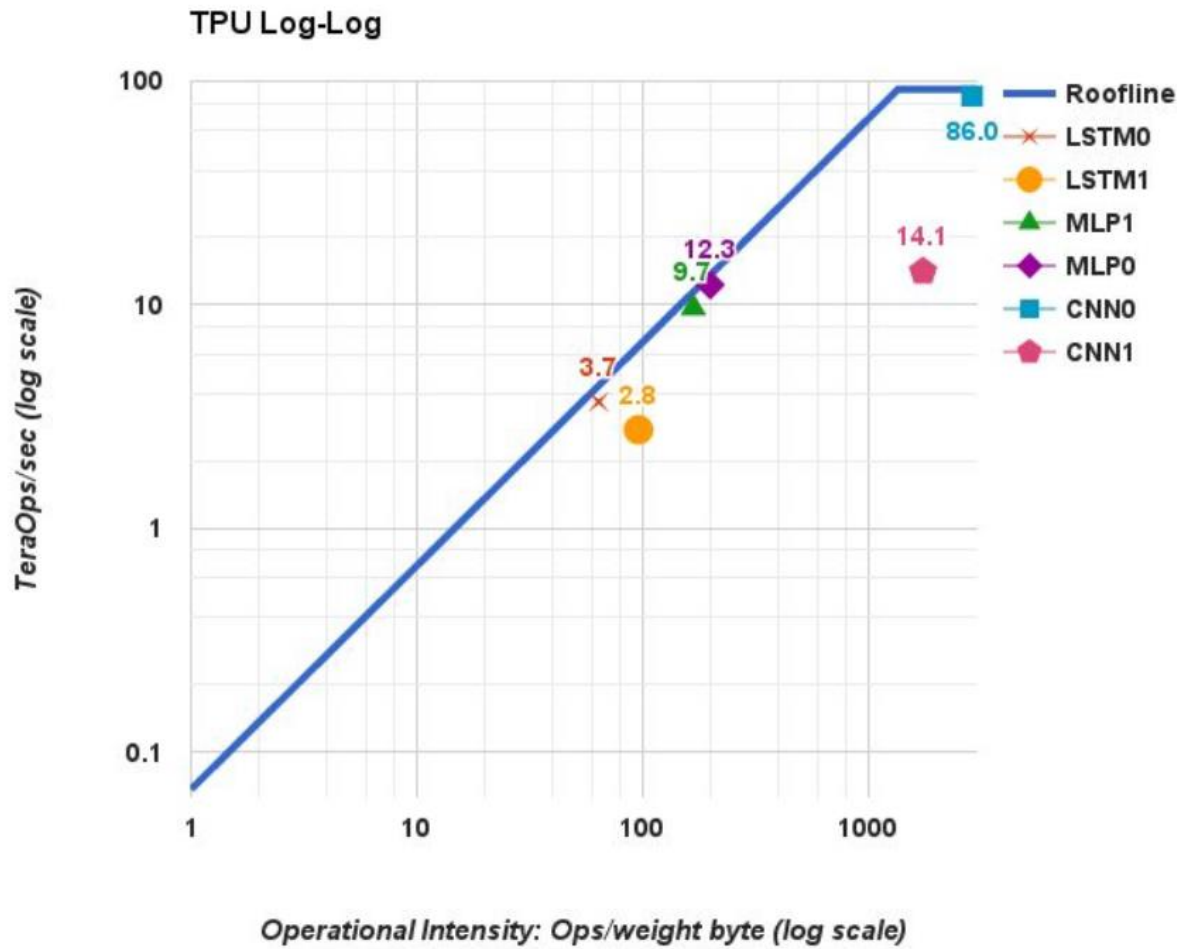
Roofline Model: NVIDIA H100



LLM decode achieves <1% of peak FLOPS. The same hardware that excels at training appears woefully underutilized during inference.

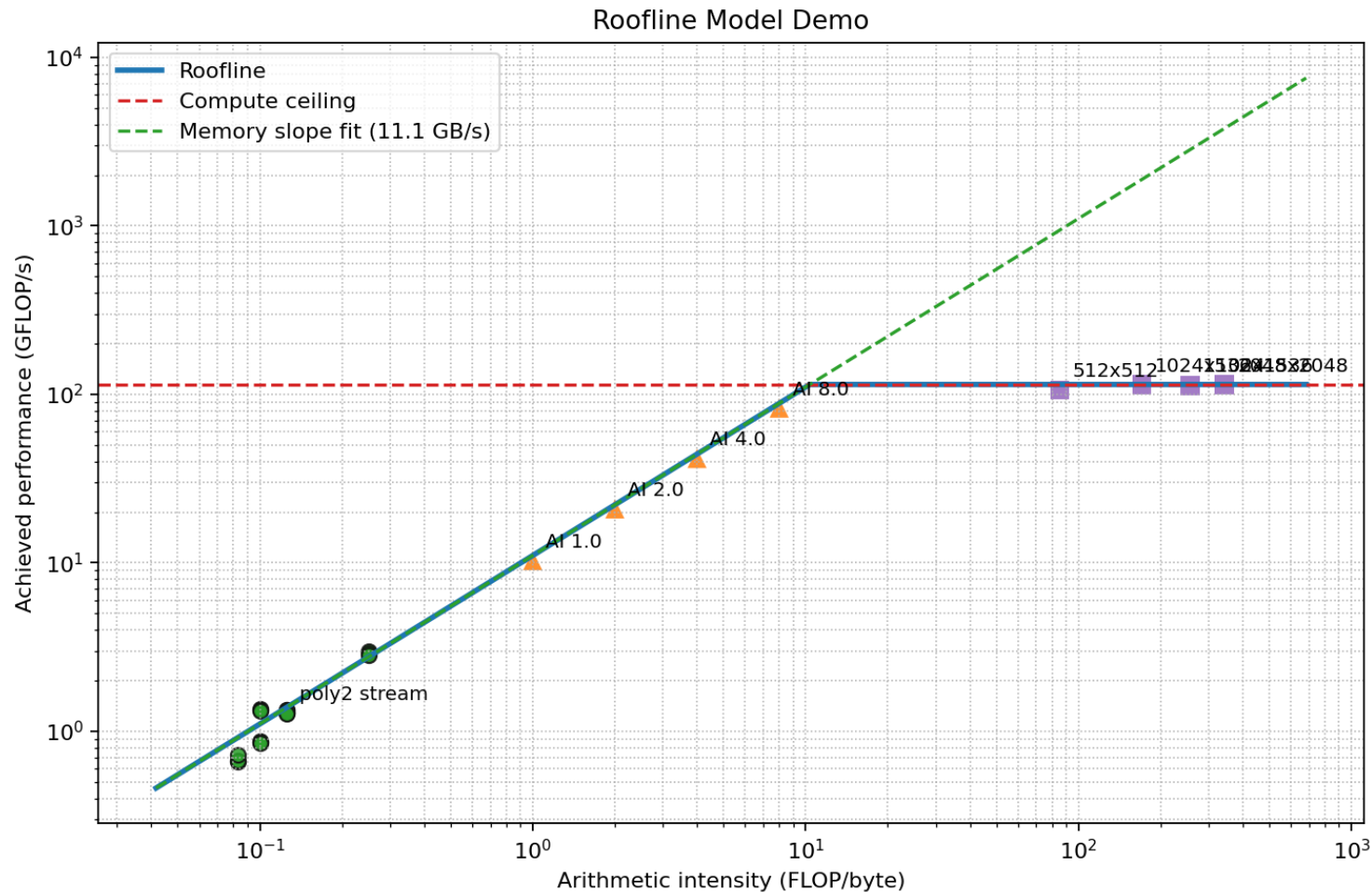
Diagnostic power: The Roofline tells you which resource to optimize before you spend money.

Roofline: TPU v1



<https://arxiv.org/abs/1704.04760>

Roofline Model of t3.large EC2



Roofline: Workload placement

Workload	/ (FLOP/B)	% Peak	Regime
LLM decode (B=1)	~ 1	$< 1\%$	Memory-bound
LLM decode (B=32)	~ 32	$\sim 5\%$	Memory-bound
CNN training	50–200	30–60%	Near ridge
LLM prefill	100–500	50–80%	Transitioning
LLM training (175B)	~ 2000	30–50%	Compute-bound

Below ridge: Buy bandwidth
Faster GPU is wasted money

Above ridge: Buy compute
Faster HBM is wasted money

Your turn: Roofline diagnosis

Diagnose the bottleneck

A 7B model (14 GB FP16 weights) served at batch size 1 on an H100

- $R_{\text{peak}} = 1,979$ TFLOPS
- $\text{BW} = 3.35$ TB/s
- Ridge point = ~ 591 FLOP/byte

Calculate the arithmetic intensity and determine the regime

Node & bandwidth

The node: Aggregating accelerators

Why node: HBM has bandwidth but not enough capacity.

Component	175B Model
Weights (FP16)	350 GB
Gradients	350 GB
Optimizer (Adam)	2.1 TB
Total	2.8 TB
Single H100 HBM	80 GB
8-GPU Node HBM	640 GB

Still not enough for full training state → ZeRO sharding, offloading

DGX H100 node:

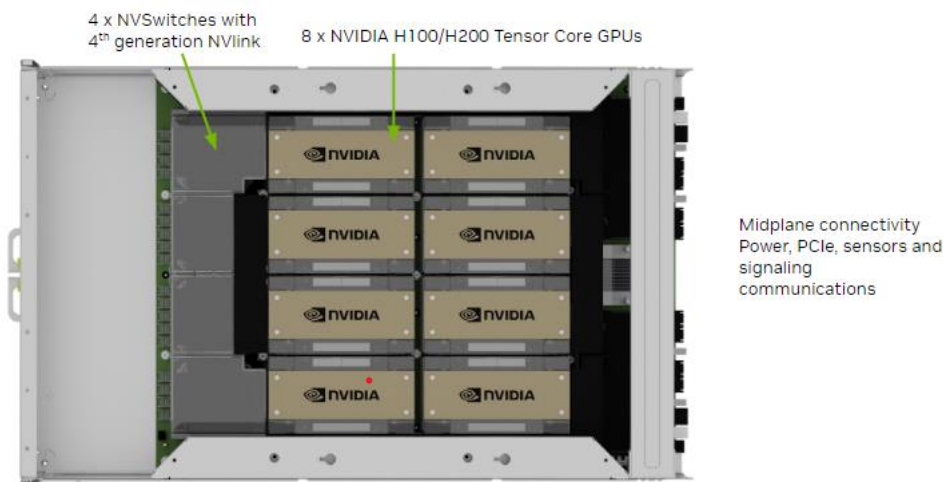
8x H100 SXM GPUs

640 GB aggregate HBM

NVSwitch: 900 GB/s per GPU

4x NVSwitch chips

Total: ~7 kW



<https://docs.nvidia.com/dgx/dgxh100-user-guide/introduction-to-dgxh100.html>

The node: Aggregating accelerators

Why node: HBM has bandwidth but not enough capacity.

Component	175B Model
Weights (FP16)	350 GB
Gradients	350 GB
Optimizer (Adam)	2.1 TB
Total	2.8 TB
Single H100 HBM	80 GB
8-GPU Node HBM	640 GB

Still not enough for full training state →
ZeRO sharding, offloading

DGX H100 node:

8x H100 SXM GPUs

640 GB aggregate HBM

NVSwitch: 900 GB/s per GPU

4x NVSwitch chips

Total: ~7 kW

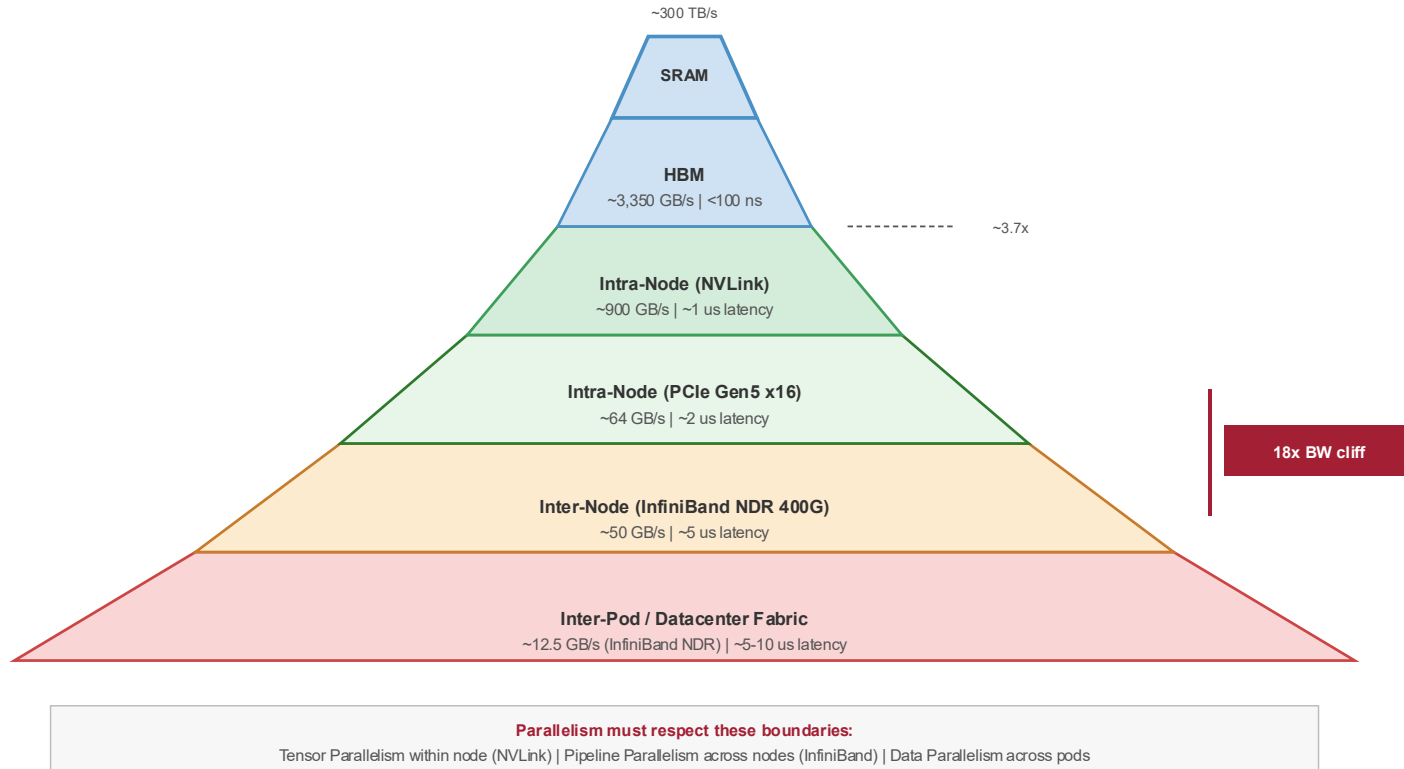
Key insight:

NVSwitch transforms 8 independent GPUs into one “**virtual accelerator**” with unified memory.

<https://docs.nvidia.com/dgx/dgxm100-user-guide/introduction-to-dgxm100.html>

The bandwidth hierarchy

The Bandwidth Hierarchy: Orders-of-Magnitude Staircase



The bandwidth hierarchy: The numbers

Domain	Interconnect	BW	Latency	Scope
Intra-Package	Si Interposer	~3,350 GB/s	<100 ns	Single chip
Intra-Node	NVLink 4.0	~900 GB/s	~1 μ s	8 GPUs
Intra-Node (IO)	PCIe Gen5 x16	~64 GB/s	~2 μ s	CPU–GPU
Inter-Node	IB NDR 400G	~50 GB/s	5–10 μ s	1000s GPUs

The 18x cliff: NVLink (900 GB/s) → InfiniBand (50 GB/s). This single boundary dictates the entire parallelism hierarchy for distributed training.

Rack & pod

The rack: Power and cooling

AI rack power density: 10x traditional datacenter!

Component	Power
32 GPUs ($\times 700$ W)	22.4 kW
CPU, NVSwitch, NICs	~ 8 kW
Power conversion losses	~ 3 kW
Total per rack	33–40 kW
Traditional server rack	5–10 kW

128 racks for 175B model training $\rightarrow$ 5 MW cluster

PUE: Cooling tax

$$\text{PUE} = \frac{\text{Total facility power}}{\text{IT equipment power}}$$

Air-cooled: PUE 1.4—1.6
(40—60% wasted!)

Liquid-cooled: PUE 1.05—1.10
(5—10% overhead only)

For 1,000 GPUs: liquid cooling saves **294 kW** $\rightarrow$ \$180K/yr in electricity

The pod: Warehouse-scale computer

1,024-node DGX H100 cluster

Resource	Scale
GPUs	8,192
Racks	~250
Power draw	7–10 MW
Weight	300+ metric tons
IB switches	Hundreds
Cables	Tens of thousands

175B training time

Physics limit: ~**12 hour**
(8,192 GPUs at **45% MFU**)

Actual: **2—4 weeks!**

(**comm, bubbles, checkpoints, failures**)

Gap: **15—25x**

Closing the gap is distributed
systems engineering

The datacenter floor must support 1,500—2,500
kg/m² (vs. 500—1,000 traditional)

Scaling efficiency

$$\eta_{scale} = \frac{T_{compute}}{T_{compute} + T_{comm} + T_{idle}}$$

GPUs	Ideal Speedup	Actual	η_{scale}
8	8×	7.6×	95%
64	64×	54×	85%
1,024	1,024×	615×	60%
8,192	8,192×	3,277×	40%

Amdahl at scale: The first 100 GPUs provide near-linear speedup. Beyond 2,000—4,000, marginal benefit per GPU approaches zero.

Your turn: Node memory budget

Memory budget: 70B model

Calculate per-node memory:

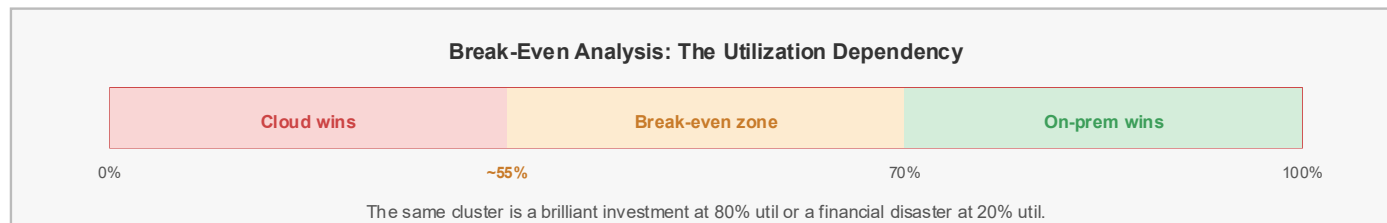
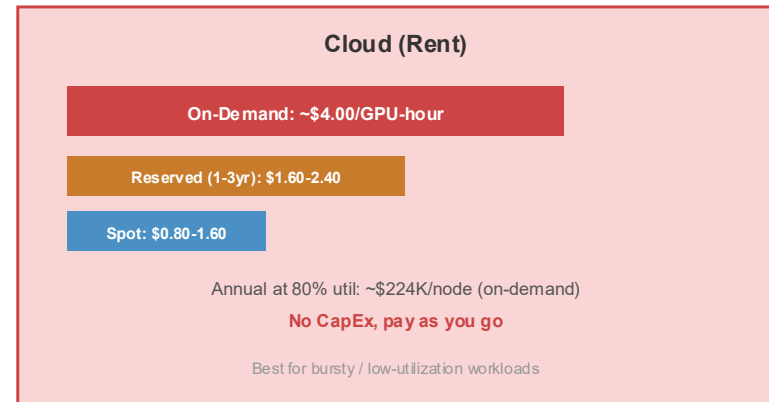
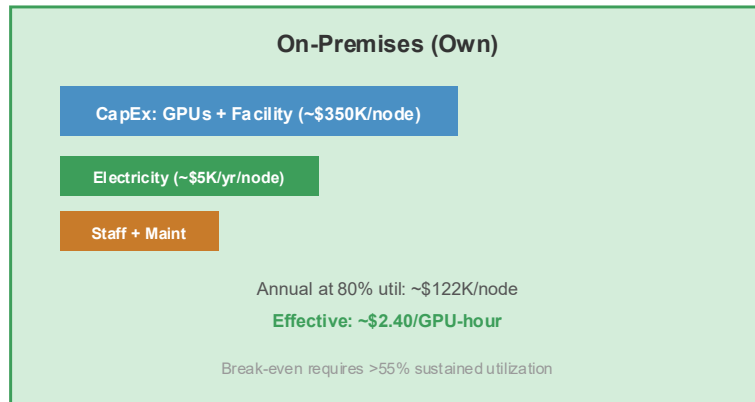
- Parameters: 70B x 2 bytes (FP16)
- Gradients: same as parameters
- Adam optimizer: $m + v + \text{FP32 copy} = 12$ bytes/param
- Activations: batch-dependent

Does it fit on one 8-GPU node (640 GB HBM)?

TCO (total cost of ownership)

Total cost of ownership

TCO: Build vs. Buy — The Utilization Question



Infrastructure economics: utilization is the single variable that flips the build-vs-buy decision.

Central tension: Utilization is the single variable that flips the build-vs-buy decision.

TCO: The numbers

	On-Premises	Cloud (On-Demand)
CapEx	\$350K/node (amortize 3 yr)	\$0
Electricity	~\$5K/yr/node	Included
Annual cost	~\$122K/node (80% util)	~\$224K/node (80% util)
Eff. rate	~\$2.18/GPU-hour	\$4.00/GPU-hour
Break-even	~43.5% sustained utilization	

Own if: continuous 24/7 workloads, predictable demand, 3+ year horizon

Rent if: bursty demand, rapid experimentation, <2 year horizon

Question: Build vs. buy

Build vs. buy decision

Your team plans to train a 70B model:

- 500 H100 GPUs for 3 weeks per run
- 4 training runs per year
- Cloud rate: \$4.00/GPU-hour
- On-prem: \$350K/node, 3-yr amortization

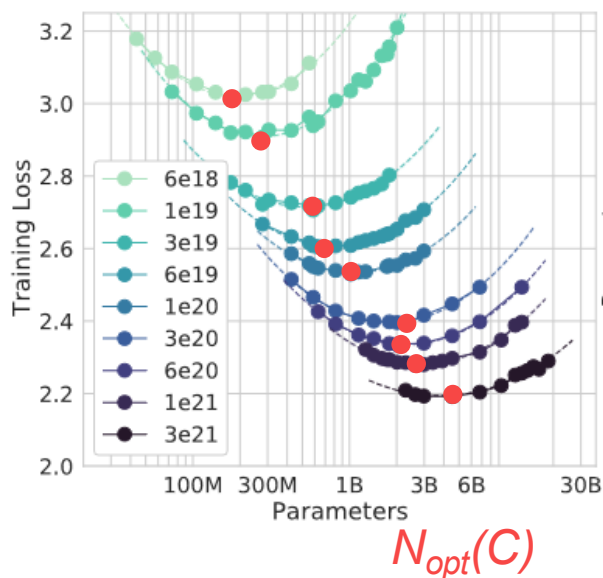
Calculate annual cost for both options

How many runs/year to justify on-prem?

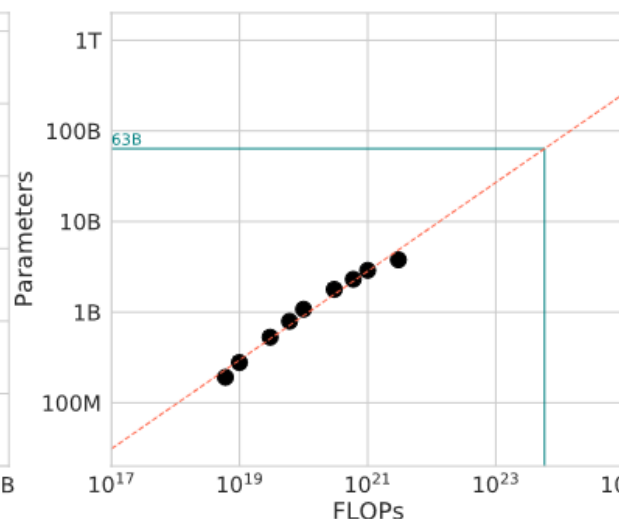
Scaling laws

Scaling laws

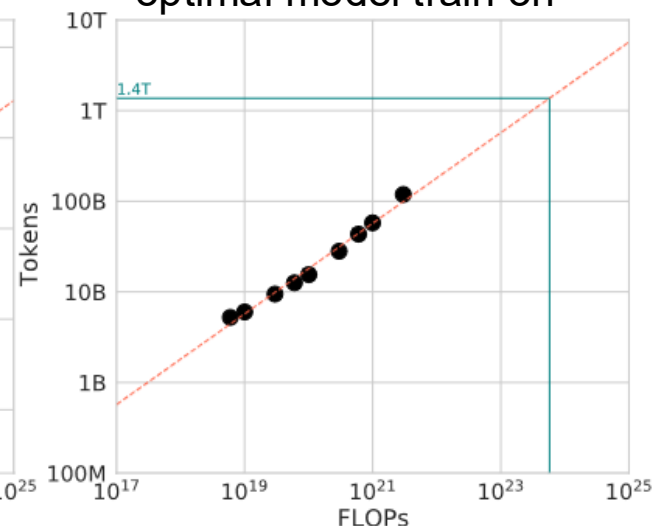
Find optimal N for each fixed compute budget



As training compute scales, how large should the optimal model be



As training compute scales, how many tokens should the optimal model train on



$$C \approx 6ND \longrightarrow N_{opt} \propto C^a \longrightarrow D_{opt} \propto C^b$$

$$\downarrow$$

$$D \propto N^{0.74}$$

<https://arxiv.org/pdf/2203.15556>

Compute-optimal scaling (Chinchilla)

Chinchilla optimality

$$D \propto N^{0.74}$$

Dataset size (D) and model size (N) must grow in coordinated proportions →

To achieve the best loss under a fixed compute budget.

Model	Params	FLOPs
GPT-1	117M	$\sim 10^{18}$
GPT-2	1.5B	$\sim 10^{19}$
GPT-3	175B	$\sim 10^{23}$
GPT-4	$\sim 1\text{T}$	$\sim 10^{25}$

10x improvement requires
100—1000x more compute.

Three resource regimes:

- **Compute-limited:** Train smaller models longer
- **Data-limited:** Train larger models fewer steps
- **Optimal:** Chinchilla frontier

<https://arxiv.org/pdf/2203.15556>

Question: Chinchilla data requirements

Chinchilla scaling exercise

Given: $D \propto N^{0.74}$

A 7B model is Chinchilla-optimal at ~140B tokens.

Calculate:

1. How many tokens for a 70B model?
2. What happens if you double compute but keep data fixed at 140B tokens?